Tenmark Audit App - Technical Handover and Data Migration Specification

Tenmark Audit App — Technical Handover & Data Migration Specification

Prepared for: Tenmark Core engineering managers **Prepared by:** Oro Corp (VJ, sole developer; technical continuity: Vivek from end of July 2026) **Purpose:** a complete, self-contained technical brief covering what this app does, exactly what data it holds today, the database migration required (Firebase Firestore → Tenmark’s PostgreSQL/MongoDB), and every piece of business logic that must survive the move. Written so your engineering team can plan and execute the integration without needing a live walkthrough for the basics.

Part 1 — Executive Summary

The Tenmark Audit App is a browser-based tool used by Oro’s audit team to physically verify gold ornaments pledged against Tenmark Capital’s (TCPL) loan book. An auditor looks up a loan, sees what Oro’s Metabase-backed loan system says should be there (weight, karat, ornament count), physically re-measures it, and records what they actually found — flagging any discrepancy (excess funding, spurious/fake items, weight mismatches).

Current architecture: static HTML/CSS/JS frontend (no framework, no build step) + Firebase (Firestore for app data, Firebase Authentication for login) + Vercel (hosts the frontend and 8 serverless API functions). Live loan data is read from Oro’s Metabase instance (never written to).

What’s changing: three things, per your team’s confirmed stack and this migration’s scope: 1. **Storage moves off Firebase Firestore** onto Tenmark’s own database (PostgreSQL via Sequelize, and/or MongoDB via Mongoose — both confirmed available on your infrastructure; a decision on which one holds this data is still open, addressed in Part 3). 2. **The frontend likely gets rebuilt** into your React 18 + TypeScript + Tailwind stack (exact approach — full rewrite vs. embedded module — still an open question with your team, addressed in Part 6). 3. **Authentication likely consolidates** into your existing Passport/JWT + Google OAuth system, replacing this app’s standalone Firebase Auth (also an open question, Part 6).

This document focuses primarily on **Part 1’s first point — the data migration** — since that’s the piece with the most concrete, gatherable specification today. Parts 6 and 7 cover what’s still genuinely undecided and needs your team’s input before further code can be written.

Part 2 — Current Data Model (Source: Firebase Firestore)

Firestore is a **NoSQL document database** — there is no schema enforcement, no foreign keys, no joins. Every fact below was extracted directly from reading the actual application code (`app.js` , `auditDataService.js` , the `api/*.js` serverless functions), not from assumption or memory. Three collections exist.

2.1 Collection: `audits`

The core collection — one document per audit event performed. **Critically: this is not one document per loan.** A single `loanId` can have multiple `audits` documents over its lifetime (every re-audit creates a new document; nothing is ever overwritten). Only the most recent document per `loanId` (by its `date` field) is considered the current, authoritative state for that loan.

Field	Type (as stored)	Nullable	Description
`id`	string (Firestore auto-ID)	No	Document identifier. Opaque, non-sequential.
`loanId`	string	No	The stable business key, e.g. `"TCGL31241368"` . **Not unique** — see above.
`date`	string, `YYYY-MM-DD`	No	The audit date. Can be backdated by the auditor. Distinct from `submittedAt` .
`auditor`	string	No	Auto-derived from the logged-in user's email at submission time (e.g. `vj@orocorp.in` → `"Vj"`) — not a foreign key to a user record.
`tw`	number	Yes	Tare weight reading, in grams. `null` on not-yet-audited placeholder records (see 2.1.1 below).
`twRecheckedBy`	string	Yes	Set only if the tare weight was rechecked after initial audit.
`twUpdatedAt`	string (ISO 8601 timestamp)	Yes	When the most recent TW save happened. This is the durable record of a recheck — see note below on `_twSubmitted` .
`excessFunding`	string enum: `"Yes"` | `"No"`	No	Whether the loan was funded for more than the audited gold value justifies.
`excessAmount`	number	Yes	Only meaningful when `excessFunding === "Yes"` .
`spurious`	string enum: `"Yes"` | `"No"`	No	`"Yes"` if any ornament in this audit was flagged as fake/substituted.
`spuriousOrnaments`	array of strings	Yes	Ornament **type names** (not IDs) flagged spurious.
`city`	string	Yes	Snapshot of the loan's city **at audit time** — not a live reference, will not reflect later changes.
`branch`	string	Yes	Same snapshot caveat as

			`city`.
`loanAmount`	number	No	The loan's disbursed amount. **Migration note:** older records created before a bug fix (mid-2026) may have this stored as a formatted currency string (e.g. `"₹1,20,000"`) instead of a number — see Part 4.3 for the exact handling required.
`loanBookingDate`	string (`YYYY-MM-DD`) or `null`	Yes	When the loan was originally booked in Oro's system.
`remarks`	string	Yes	Freeform auditor notes. Legitimately blank most of the time.
`newPacketId`	string	Yes	Freeform; blank means unchanged from the original packet.
`ornaments`	array of objects	No	Nested array — see 2.1.2 below for the full sub-schema.
`submittedAt`	string (ISO 8601 timestamp)	No	The real save timestamp, distinct from the (possibly backdated) `date`.
`source`	string, literal `"metabase-sync"`	Yes	Only present on auto-generated placeholder records — see 2.1.1.
`syncedAt`	string (ISO 8601 timestamp)	Yes	Only present alongside `source`.

2.1.1 — Placeholder ("pending") records

A background sync process creates a placeholder `audits` document (with `source: "metabase-sync"`, `tw: null`, and no real audit fields populated) for every loan that becomes active in Oro's system but hasn't yet been physically audited. **These are excluded from every calculation and display in the app** — every summary count, filter, and table explicitly filters `source !== "metabase-sync"`. They exist purely as a "this loan is known but not yet audited" bookkeeping record.

Migration note: this placeholder-creation mechanism is a candidate for being dropped entirely during migration, since (a) nothing reads it as meaningful data today, and (b) the app already recomputes "which loans need auditing" live, on every page load, by querying the live loan data directly and diffing against real audit records — it does not and has never depended on these placeholders. Confirm with Oro's team before deciding either way, but do not assume this mechanism needs replicating without discussing it — see Part 7, item 3.

2.1.2 — Nested object: `ornaments[]`

Each `audits` document contains an array of these. This is the part of the schema most affected by a NoSQL → SQL migration decision (Part 3 discusses this directly).

Field	Type	Nullable	Description
`type`	string	No	Ornament category label (e.g. `"Chain"`, `"Ring"`). **A label, not a stable identifier** — confirmed to drift over time in the source

			system (e.g. `"Stud"` was later renamed `"Studs"`). Do not treat as a reliable join key across time.
`goldId`	string	Yes	A stable ID from the source loan system, when available. **This is the reliable identifier** for matching the same physical item across multiple audits — preferred over `type` whenever present. Older records may lack it.
`count`	number	No	Original piece count for this ornament line, as recorded by Oro's ops team.
`countAudit`	number	Yes	Auditor's own recorded piece count. **Known gap:** some older imported records have `count` but not `countAudit` — any consumer must fall back to `count` when `countAudit` is absent, never treat a missing value as zero.
`gw`	number	No	**Gross weight — a TOTAL for this entire line, never a per-piece figure.** This is a hard, load-bearing rule throughout the entire app and its source system. If a line has `count: 2`, `gw` is already the combined weight of both pieces. **Never multiply or divide this by count.** A real historical bug (since fixed) resulted from violating exactly this rule.
`gwAudit`	number	Yes	Auditor-measured gross weight, same total-not-per-piece rule applies.
`gwPC`	number	Yes	Pre-computed gross-weight-per-piece (a derived display convenience, computed once at write time — not re-derived on every read).
`karat`	number	No	Original recorded karat/purity.
`karatAudit`	number	Yes	Auditor-measured karat.
`karatPC`	number	Yes	Derived per-piece karat.
`nw`	number	No	Net weight. **Formula:** `(GW − Stone Deduction) × (Karat / 22)`.
`nwAudit`	number	Yes	Same formula applied to audited figures.

`nwPC`	number	Yes	Derived per-piece net weight.
`stoneDed`	number	No	Original stone/non-gold deduction weight.
`stoneDedAudit`	number	Yes	Auditor-measured stone deduction.
`stoneDedPC`	number	Yes	Derived per-piece stone deduction.
`hallmark`	string	Yes	Hallmark reading, freeform.
`spurious`	string enum: `"Yes"` | `"No"`	No	Per-ornament flag; rolls up into the parent audit's `spurious` / `spuriousOrnaments` fields.

2.2 Collection: `app_settings`

Exactly one document, with a hardcoded ID of `"config"`. Global configuration, not per-user data.

Field	Type	Nullable	Description
`pendingDays`	number	No	Configurable "audit is due" cycle length, in days. Manager-editable.
`twThreshold`	number	No	Tare weight mismatch flagging threshold, in grams. Default `0.3`.
`settingsPassword`	string	No	**A single shared password**, not a per-user credential, gating access to the Settings screen. This is application data (stored in the database), not an environment/deployment secret. **Flag for your team:** if this app's login is replaced by your dashboard's own auth, this entire concept likely needs rethinking — a shared password is a weaker access model than per-user roles, and exists today only because building full per-screen RBAC wasn't in scope originally.
`branches`	array of strings	No	Manager-registered branch names, used for filtering.
`updatedAt`	string (ISO 8601)	No	Last settings-save timestamp.
`lastSyncAt`	string (ISO 8601)	Yes	Last time the loan-sync process ran (see 2.1.1 — this process may not survive migration).
`lastSyncStatus`	string enum: `"success"` | `"completed_with_errors"`	Yes	Displayed as a manager-only banner on login if the last sync had failures.

`lastSyncFailureCount`	number	Yes	Count backing the banner above.

2.3 Collection: `users`

One document per user account. **Document ID = Firebase Auth UID** (not email, not a sequential integer).

Field	Type	Nullable	Description
`email`	string	No	Currently restricted to the `@orocorp.in` domain by convention (not enforced at the database level — enforced by whichever process creates the account).
`role`	string enum: `"auditor"` | `"manager"`	No	The only permission distinction in the app today. No finer-grained or city/branch-scoped permissions exist.
`uid`	string	No	Duplicate of the document ID, stored as a field as well — redundant, but the application code relies on reading it as a field in at least one place; preserve this duplication if porting logic as-is, or refactor the one call site if cleaning this up.

Flag for your team, high priority: if your dashboard's own login (Passport JWT + Google OAuth, per your team's stack) replaces this app's authentication entirely — which is the direction currently being discussed — **this entire collection, and its three supporting backend endpoints (create-user, remove-user, reset-password), may not need to be migrated at all.** Confirm this decision before spending engineering time porting user-management logic that may simply be deleted. See Part 6.

Part 3 — Migration Target: Choosing Between PostgreSQL and MongoDB

Your infrastructure runs both (PostgreSQL via Sequelize as primary, MongoDB via Mongoose as secondary). This is a real decision your team needs to make, not something this document can decide unilaterally — but here is the concrete tradeoff, with working draft schemas already built for **both** options so neither path requires starting from zero.

3.1 The core tension: the `ornaments[]` array

Firestore stores `ornaments` as a nested array inside each audit document — a natural fit for a document database. Your two target databases handle this very differently:

Option A — PostgreSQL, using a JSONB column. Store `ornaments` as a single JSONB column on the `audits` table, exactly mirroring the current nested-array shape. This is the **lowest-risk, most direct migration path** — the data doesn't need restructuring, just a change of storage engine. The tradeoff: you lose the ability to run SQL-level queries or joins against individual ornament fields (e.g. "find all audits containing a Ring over 20g" would require JSONB query operators, not a simple `WHERE` clause on a normal column).

Option B — PostgreSQL, fully normalized. A separate `ornaments` table with a foreign key back to `audits`, one row per ornament. This enables real SQL queries at the ornament level, at the cost of a genuinely bigger migration script and a schema redesign, not just a storage-engine swap.

Option C — MongoDB, using an embedded sub-document. Store `ornaments` as an array of embedded documents inside each `audits` document — structurally almost identical to the current Firestore shape. This is arguably the **closest possible match** to what exists today, if your team decides Mongo is the right home for this specific dataset (as opposed to Postgres, which is likely home to more transactional/relational business data elsewhere in Tenmark Core).

My recommendation, as a starting point for discussion, not a final decision: Option A (Postgres + JSONB) if audit data needs to sit alongside other relational Tenmark Core data and be queried via SQL for reporting; Option C (Mongo) if this dataset is going to remain largely self-contained and queried by application code rather than ad-hoc SQL. Option B should only be chosen if there's a concrete, near-term need to query inside ornament data at the SQL level — it's the most expensive option for a benefit that doesn't yet have a stated use case.

3.2 Draft schemas — already built and tested

Both options have been built out as real, runnable code (not just diagrams) and validated:

- **`audit.model.js`** — a complete Sequelize model for Option A, with every field from Part 2.1 mapped to a Postgres column type. Tested against a real (in-memory) SQLite database: confirmed it correctly stores and retrieves a sample audit with nested ornaments, confirmed multiple audits per `loanId` are correctly NOT blocked by an accidental unique constraint, confirmed round-trip data integrity.
- **`migrations-create-audits.js`** — the corresponding Sequelize migration file (in your team's own `sequelize-cli` up/down format), verified to run and roll back cleanly, and cross-checked field-for-field against the model file to confirm zero drift between the two.
- **`audit.schema.js`** — a complete Mongoose schema for Option C, with the same field mapping. Validated with real sample documents, including three negative tests (missing required field, invalid enum value, incomplete embedded ornament) that all correctly failed validation — proving the schema actually enforces its rules rather than silently accepting bad data.

All three files are included in this handover package under `code - but not deployed anywhere - reference/`. None are wired into anything running — they are ready-to-review drafts for your team's engineers to critique, adjust, and adopt.

3.3 What must be preserved regardless of which option is chosen

1. **Multiple documents/rows per `loanId` must remain possible.** Do not add a unique constraint on `loanId` alone. The "which one is current" logic (most recent by `date`) is application-layer logic today (`computeDedupedAudits()` in `app.js`) and should either be replicated in the new application layer or expressed as a database view/query — but the underlying data must retain full re-audit history.
2. **`gw` / `gwAudit` must remain line-totals, not per-piece figures**, in whatever storage format is chosen. This is a business rule, not a storage detail, and no column type enforces it — it must be enforced by whatever code reads/writes this field going forward.
3. **`loanAmount` must be migrated as a genuine numeric type**, with pre-migration data cleaning for older records that may still hold a formatted currency string (see Part 4.3).

Part 4 — Data Migration Logic: Field-by-Field Transformation Rules

This section is the actual migration script's specification — what needs to happen to get from a Firestore export into the new database, correctly.

4.1 Extraction

Firestore data can be exported via Google Cloud's Firestore export tool (produces files in a GCS bucket, in a format Google's own `firestore-export` or similar tooling can parse) or read directly via the Firestore REST API / Admin SDK, paginated. Given the dataset size (a few hundred to low thousands of audit records, based on current usage), a direct read-and-transform script is entirely practical — no need for Google's bulk export tooling for a dataset this size.

4.2 Document ID handling

Firestore's auto-generated document IDs are opaque strings (e.g. `a1B2c3D4e5F6`) — not sequential, not meaningful. Decide on a new primary key strategy for the target database: - **If UUID:** generate a fresh UUID per record at migration

time — do not attempt to reuse or reformat the Firestore ID as a UUID. - **If auto-increment integer:** likewise, generate fresh sequential IDs; the old Firestore ID becomes irrelevant after migration (optionally preserve it in a `legacyFirestoreId` column for one-time debugging/audit-trail purposes during the transition period, then it can be dropped later).

4.3 The `loanAmount` type-cleaning step — mandatory, not optional

This is the single most important data-cleaning step in the entire migration, because it will otherwise cause the migration script to fail outright (if the target column is a strict numeric type) or silently corrupt data (if it isn't).

The problem: a bug (since fixed in the source application, but not retroactively fixed in historical data) caused `loanAmount` to sometimes be saved as a formatted currency string, e.g. `"₹1,20,000"`, instead of a plain number `120000`. This affects an unknown subset of older records — every record created via the app's manual "New Audit" flow before the fix, but not records created via the automatic loan-sync placeholder mechanism, which always used a plain number.

The required transformation, to be applied to every record during migration:

```
IF loanAmount is already a number → use as-is
IF loanAmount is a string → strip every character that is not a digit or a decimal point, then parse the result as
a number
```

This exact stripping logic (`String(value).replace(/[^0-9.]/g, '')`) is already implemented and tested in the current application's own display code — it's the same logic that lets old and new records display correctly side-by-side today. Reuse it verbatim in the migration script rather than reinventing it.

Validation step, mandatory: after transformation, spot-check that no record ends up with an implausible value (e.g. `0`, or a wildly different number than before) — a currency string with an unexpected format (e.g. a stray currency symbol this regex doesn't anticipate) could silently produce a wrong number rather than an obvious error. Compare a sample of before/after values manually before trusting the full migration run.

4.4 The AP-valuation vs. Maker-valuation distinction — not a Firestore field, but critical context

This doesn't appear as a field to migrate, but it's essential context for understanding why the numbers in `audits` documents look the way they do: every ornament figure that originated from Oro's live loan system (via Metabase) — weight, karat, count — reflects the **AP (Appraisal Partner) valuation specifically**, never the Maker/Checker's independent re-verification of the same physical items. Oro's source system stores both valuations for every loan; the application explicitly filters for the AP one everywhere it reads gold data. This is a deliberate business decision (not a technical default), and if Tenmark Core's own systems ever need to cross-reference this audit data against Oro's live loan data again in the future, whoever does so must apply the same AP-only filter, or the numbers won't reconcile.

4.5 Timestamps and timezone handling

All timestamps in the current system (`submittedAt` , `twUpdatedAt` , `syncedAt`) are stored as ISO 8601 UTC strings. **Known bug, not yet fixed in the source app:** the "is this today?" calculation used for daily counters compares against the browser's UTC date directly, without adjusting for IST (UTC+5:30) — meaning for roughly 5.5 hours every night (midnight to 5:30 AM IST), a record saved in that window can appear to belong to "yesterday" rather than "today." **When migrating and when building any new reporting logic on the new database, compute "today" in IST explicitly, not by taking the UTC date directly** — don't carry this bug forward into new code even though it exists in the old.

4.6 What does NOT need migrating

- The `users` collection and its 3 supporting endpoints — **only if** your dashboard's auth replaces this app's login entirely (Part 6, still unconfirmed).
- The `source: "metabase-sync"` placeholder records (Part 2.1.1) — these carry no information not already recomputable live from Oro's loan system; migrating them is optional bookkeeping at best, not a real requirement.
- `app_settings.settingsPassword` — only if the Settings-access model is being redesigned as part of the auth consolidation.

Part 5 — Business Logic That Must Be Re-Implemented, Not Just "Migrated"

A database migration moves data. It does not, by itself, move the *rules* that keep that data meaningful. The following logic currently lives in application code (`app.js`) and must be deliberately reimplemented in whatever new application layer reads/writes this data — none of it is enforced by the database schema itself, in either the old or new system.

5.1 "Which audit is current for this loan" — deduplication

Given multiple `audits` records for the same `loanId`, the current record is whichever has the most recent `date`. This logic must exist somewhere — either as application code (as it is today, in `computeDedupedAudits()`), or as a database view/query (`SELECT ... WHERE date = (SELECT MAX(date) ... GROUP BY loanId)` or equivalent). **Do not assume a "latest" flag or a simple "get the last inserted row" approach is equivalent** — re-audits can legitimately be entered out of chronological order relative to insertion time in edge cases (e.g. backdating), so the comparison must be on the `date` field's value, not insertion order.

5.2 Re-audit ornament matching — deliberately conservative, do not "improve" this

When an auditor re-audits a loan that's been audited before, the app tries to show what was recorded last time as a labeled reference (never a silent autofill). The matching algorithm, in priority order:

1. **Exact match by `goldId`** — used whenever both the current and a past ornament have this field populated. Unambiguous.
2. **Unambiguous match by `type` name** — for records with no `goldId` : if exactly one past ornament shares this type, use it (no ambiguity exists).
3. **Fallback match by recorded weight** — if the type name itself has drifted (a real, confirmed occurrence in the source system) but exactly one past ornament shares the recorded weight, treat this as a legitimate re-identification.
4. **If none of the above resolve to exactly one confident candidate — the system deliberately does not guess.** It shows all ambiguous candidates as a labeled list and leaves the matching field blank, rather than risk silently feeding a wrong number into a new audit.

This conservatism is intentional and load-bearing for data trustworthiness. Any reimplementation must preserve step 4's refusal to guess — this is not a gap to "fix" by making the matching more aggressive.

5.3 Ledger/counter calculations

Both the "Tare Weight remaining/completed today" counters and the "All Audits" summary cards (total/excess/spurious/clean counts) are computed by filtering and counting the deduplicated audit set (5.1) — never by a raw `COUNT()` over all `audits` records, since that would double-count loans with multiple audit records. Already extracted into pure, framework-agnostic functions in the current codebase (`computeTWCounters()` , `computeAllAuditsSummaryCounts()`) specifically so this logic can be lifted into a new frontend without needing to be re-derived from scratch — see the reference code included in this handover package.

5.4 Access control model (current state, pending your team's decision)

Enforced today via Firestore Security Rules (included in this package as `firestore.rules`), not application code — meaning the rules below are enforced at the database layer, not just the UI: - Any authenticated session (including an anonymous "guest" session) can **read** `audits` and `app_settings` . - Only a verified `manager` or `auditor` role can **write** to `audits` . - Only a verified `manager` role can **write** to `app_settings` . - **Writes to `users` are restricted to a single backend service account** — no user, including managers, writes to this collection directly; it's mediated entirely through 3 backend endpoints that verify the caller's role before acting.

Whatever database and backend replace Firestore must implement equivalent access control **at the application/API layer**, since the new database (Postgres/Mongo) has no native equivalent to Firestore Security Rules — this logic cannot simply be "turned off" without a replacement; it must be deliberately rebuilt, likely as Express middleware.

Part 6 — Open Decisions Requiring Your Team's Input

These are not gaps in this document — they are genuine open questions that only your team can answer, listed here so nothing proceeds on an unconfirmed assumption:

1. **Database choice:** Postgres (JSONB or normalized) vs. MongoDB for the `audits` data — Part 3.
2. **Frontend integration approach:** full rewrite into your React/TypeScript stack vs. an embedded module (iframe/micro-frontend) — note that your confirmed build tool (Create React App + craco, unejected) does not natively support Module Federation, which is a real technical constraint on the "embedded module" option, not just a preference.
3. **Authentication consolidation:** does your Passport/JWT + Google OAuth system replace this app's Firebase Authentication and the `users` collection entirely? This single decision determines whether Part 2.3 and its supporting endpoints need migrating at all.
4. **Database access model:** does this app's backend get direct database credentials, or does it go through an API layer your team exposes?
5. **Hosting:** do the 8 backend API functions get ported into your Express/Docker/PM2 infrastructure, or remain on Vercel and call your database/API remotely?

A full, itemized list of technical questions for your engineering team (phrased so they're answerable directly from your own codebase, without needing product/policy context) is included separately in this handover package as `tenmark-integration-questions-consolidated.md`.

Part 7 — What's Included in This Handover Package

File/Folder	Contents
`PROJECT_CONTEXT.md`	A briefing document (originally written so an AI coding assistant could onboard onto this codebase with zero prior context) — doubles as a concise technical orientation for a human engineer too.
`README.md`	The original project README — architecture, environment variables, deployment process, and a "hard-won domain knowledge" section covering bugs and fixes discovered during development.
`documentation/tenmark-data-dictionary.md`	The full field-by-field data dictionary this handover document's Part 2 was built from.
`documentation/tenmark-api-contracts.md`	Exact input/output specification for all 8 backend API endpoints.
`documentation/tenmark-env-vars.md`	Every environment variable/secret the current app depends on.
`code - but not deployed anywhere - reference/`	Draft, tested (but not live) code: TypeScript interfaces, Sequelize model + migration, Mongoose schema, Joi validation schemas — ready for your engineers to review and adapt.
`tests/`	The application's existing test suite (156+ passing assertions) — demonstrates, with runnable proof, the exact business rules described in Part 5.
`firestore.rules`	The current database-layer access control rules referenced in Part 5.4.
The full application source (`app.js` , `api/*.js` , `auditDataService.js` , `index.html` , `style.css`)	The actual running application, for direct reference alongside this document.

Recommended reading order for your engineering team: this document first (for the migration-specific plan), then

`PROJECT_CONTEXT.md` (for broader app context), then the data dictionary and API contracts docs as detailed references while actually writing migration code.